



BLOCKCHAIN TECHNOLOGIES 2

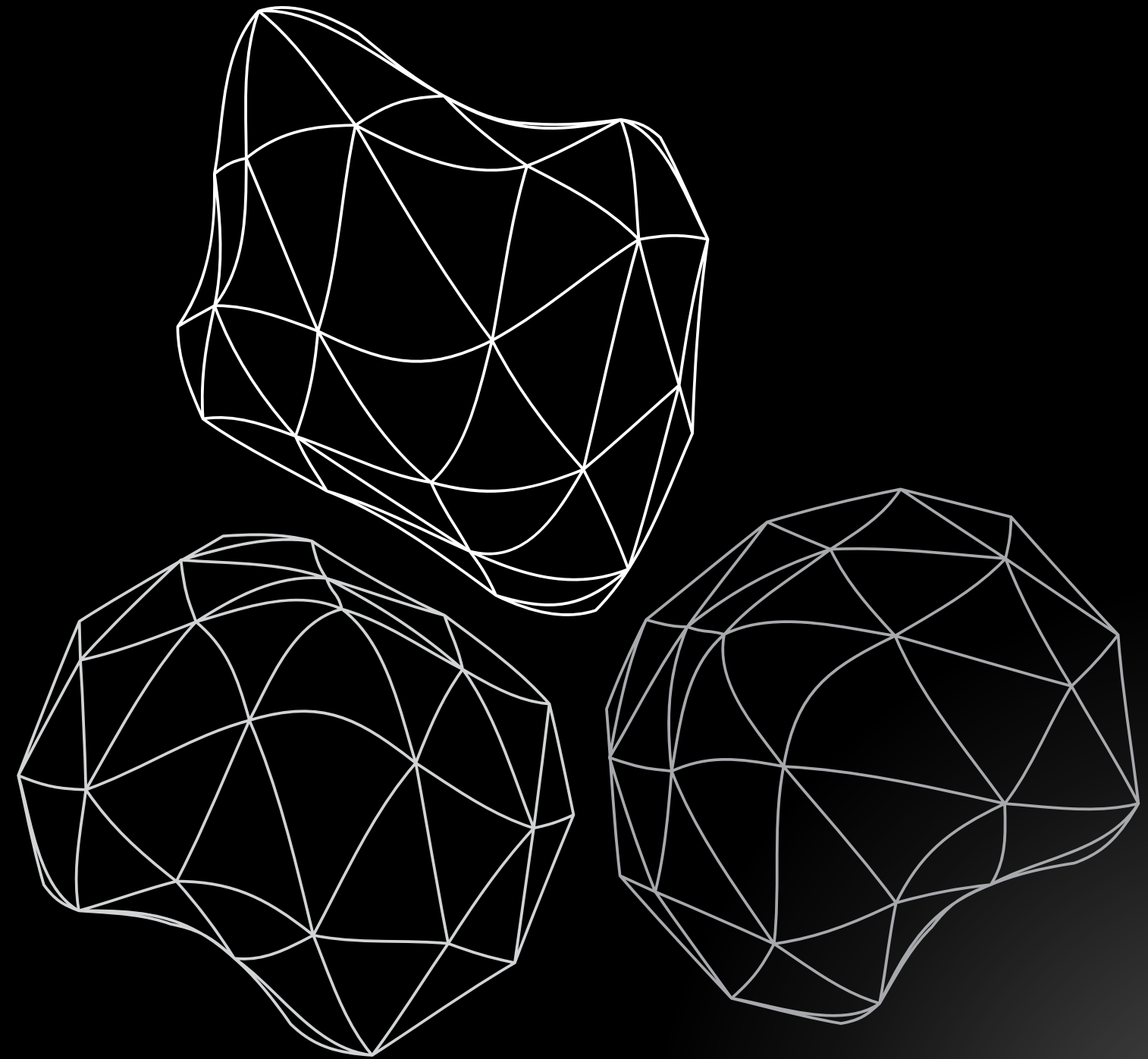
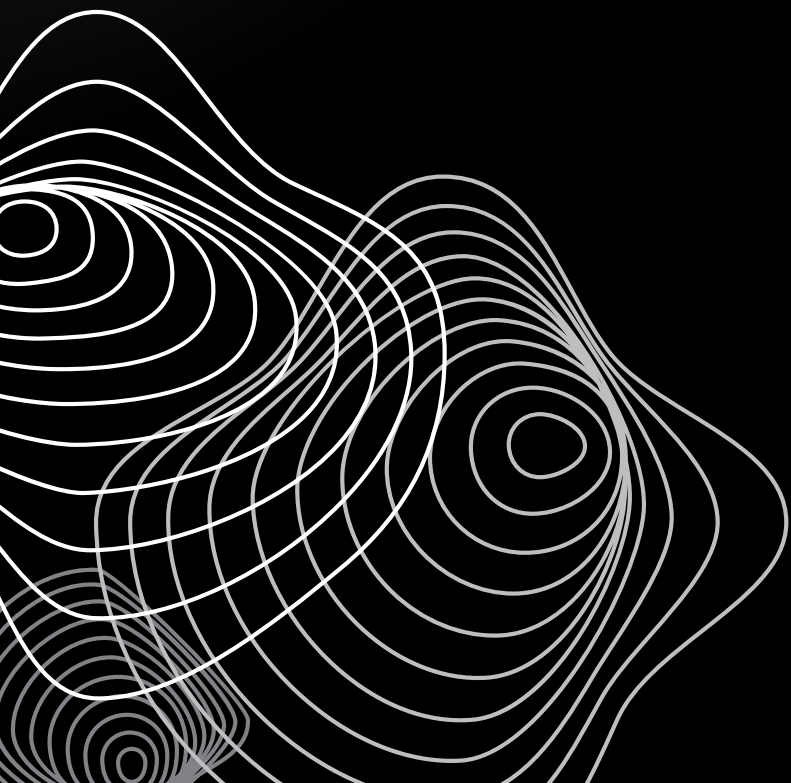
FINAL PROJECT

prepared by: Adilet Zhambyl, Aksungkar Ganiyatov and Zhaskanat Suienishuly

ON-CHAIN PREDICTION MARKET DAO

Full-Stack Decentralized Protocol

- ERC20Votes Governance
- UUPS Upgradeability
- ERC4626 Vault
- Chainlink Oracle
- The Graph Integration
- React Frontend + CI/CD



ON-CHAIN PREDICTION MARKET DAO

← → ↺

localhost:5173

🔍 ☆ 🔥 📄 🚀 🌐 🗂️ 👤

Prediction Market DAO

0xf39Fd6e51aad88F6F4ce6aB8827279cFfFb92266

Disconnect

Governance Token

Connected wallet: 0xf39Fd6e51aad88F6F4ce6aB8827279cFfFb92266

Chain ID: 31337

Token contract: 0xc6e7DF5E7b4f2A278906862b61205850344D4e7d

Balance: 10000000 PDAO

Voting Power: 10000000

Delegate: 0xf39Fd6e51aad88F6F4ce6aB8827279cFfFb92266

Delegate to myself

Governance

Governor contract: 0x4ed7c70F96B99c776995fB64377f0d4aB3B0e1C1

Create test proposal

1084655147278504001954989567271363774618067528424010470378938679310339

Proposal State: Active

Vote FOR Vote AGAINST Abstain

Fee Vault

Vault contract: 0x4A679253410272dd5232B3F7cF5dbB88f295319

Collateral token: 0x3Aa5ebB10DC797CAC828524e59A333d0A371443c

Your collateral balance: 990

Your vault shares: 10

Total vault assets: 10

10 Approve Deposit Redeem

- wallet connected
- governance section visible
- vault visible

Blockchain-Final Public

👁 Watch 0 🍴 Fork 0 ☆ Star 0

👤 master 1 Branch 0 Tags

🔍 Go to file T Add file <> Code

🌐 shaidyo docs: add architecture, audit, deployment, frontend, gas and subgraph... 1578ec8 · 1 hour ago 28 Commits

📁 .github/workflows	fix(ci): allow slither informational findings in pipeline	2 hours ago
📁 docs	docs: add architecture, audit, deployment, frontend, gas and ...	1 hour ago
📁 frontend	ci: add full contracts frontend and subgraph pipeline	2 hours ago
📁 lib	feat: initialize Foundry project with OZ v5, Chainlink, and forg...	15 hours ago
📁 script	fix(ci): format contracts and build ABIs before subgraph code...	2 hours ago
📁 src	feat(integration): add frontend dApp, subgraph indexing, and...	5 hours ago
📁 subgraph	ci: add full contracts frontend and subgraph pipeline	2 hours ago
📁 test	fix(ci): format contracts and build ABIs before subgraph code...	2 hours ago
📄 .gitignore	chore(devops): add gitignore and clean generated artifacts	4 hours ago
📄 ARCHITECTURE.md	docs: add architecture, audit, deployment, frontend, gas and ...	1 hour ago
📄 AUDIT.md	docs: add architecture, audit, deployment, frontend, gas and ...	1 hour ago
📄 COVERAGE.md	docs(coverage): add coverage report - 96.59% line, 96.38% st...	10 hours ago
📄 README.md	feat(integration): add frontend dApp, subgraph indexing, and...	5 hours ago
📄 docker-compose.yml	feat(integration): add frontend dApp, subgraph indexing, and...	5 hours ago
📄 foundry.toml	feat(integration): add frontend dApp, subgraph indexing, and...	5 hours ago

About

No description, website, or topics provided.

📖 Readme 📂 Activity ☆ 0 stars 👁 0 watching 🍴 0 forks

Report repository

Releases

No releases published [Create a new release](#)

Packages

No packages published [Publish your first package](#)

Contributors 3

👤 shubh2294 Shubham Mishra

🌐 shaidyo

🧑‍🎓 ganiyatov21

our repository

SYSTEM ARCHITECTURE

Frontend (React + Wagmi)



- Smart Contracts
- GovernanceToken
- PredictionGovernor
- TimelockController
- Treasury
- MarketFactory
- PredictionMarketV1/V2
- FeeVault
- OracleResolver



External Infrastructure

- Chainlink
- The Graph

CORE FEATURES

- DAO Governance
- UUPS Upgradeability
- ERC1155 Outcome Shares
- ERC4626 Vault Accounting
- CREATE2 Deployment

SYSTEM ARCHITECTURE

```
contract PredictionMarketV1 is UUPSUpgradeable, OwnableUpgradeable, ReentrancyGuardTransient {  
    using SafeERC20 for IERC20;
```

PredictionMarketV1 inheritance structure with UUPS upgradeability and reentrancy protection

```
contract PredictionGovernor is  
    Governor,  
    GovernorSettings,  
    GovernorCountingSimple,  
    GovernorVotes,  
    GovernorVotesQuorumFraction,  
    GovernorTimelockControl  
{
```

Governor contract architecture using OpenZeppelin governance modules

GOVERNANCE SYSTEM

DAO GOVERNANCE

IMPLEMENTED FEATURES

- ERC20Votes token
- Proposal creation
- Voting system
- Timelock execution
- Proposal lifecycle tracking

Governance Flow

Propose



Vote



Queue



Execute

Security

- Snapshot voting
- Timelock protection
- Quorum requirements

GOVERNANCE SYSTEM



```
function votingDelay() public view override(Governor, GovernorSettings) returns (uint256) {  
    return super.votingDelay();  
}
```

```
function votingPeriod() public view override(Governor, GovernorSettings) returns (uint256) {  
    return super.votingPeriod();  
}
```

```
function proposalThreshold() public view override(Governor, GovernorSettings) returns (uint256) {  
    return super.proposalThreshold();  
}
```

```
function quorum(uint256 blockNumber) public view override(Governor, GovernorVotesQuorumFraction) returns (uint256) {  
    return super.quorum(blockNumber);  
}
```

Governor configuration with voting delay, voting period, and quorum logic

```
contract GovernanceToken is ERC20, ERC20Permit, ERC20Votes, Ownable {  
    uint256 public constant MAX_SUPPLY = 100_000_000e18; // 100 million PDAO  
  
    event Minted(address indexed to, uint256 amount);  
  
    constructor(address initialOwner) ERC20("PredictDAO", "PDAO") ERC20Permit("PredictDAO") Ownable(initialOwner) {  
        _mint(initialOwner, 10_000_000e18); // 10 % initial distribution  
    }  
}
```

GovernanceToken implementation using ERC20Votes and ERC20Permit

Governance

Governor contract: 0x4ed7c70F96B99c776995fB64377f0d4aB3B0e1C1

Create test proposal

1084655147278504001954989567271363774618067528424010470378938679310339

Proposal State: Active

Vote FOR

Vote AGAINST

Abstain

DAO proposal lifecycle with active governance states

PREDICTION MARKET + VAULT

PREDICTION MARKET

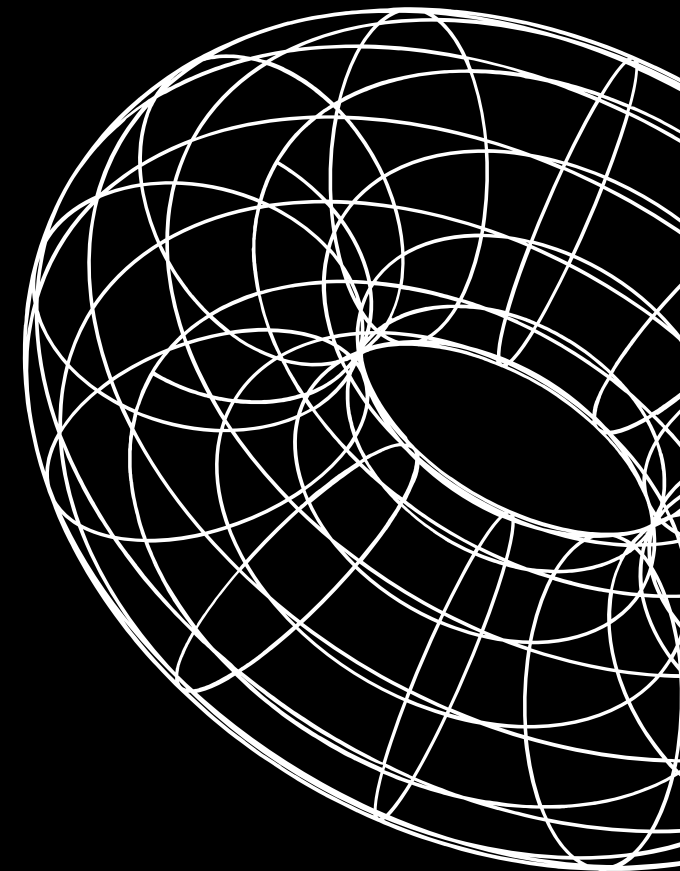
- CPMM AMM model
- Binary YES / NO markets
- ERC1155 outcome shares
- Slippage protection

VAULT

- ERC4626 accounting
- LP fee distribution
- Deposit / redeem support

UPGRADEABILITY

- UUPS Proxy pattern
- Storage-safe upgrades
- V1 → V2 upgrade path



PREDICTION MARKET + VAULT

```
contract FeeVault is ERC4626, Ownable {  
    /// @notice Only this address may push fees into the vault.  
    address public feeCollector;  
  
    event FeeCollectorUpdated(address indexed previous, address indexed next);  
    event FeesReceived(address indexed from, uint256 assets);  
  
    error NotFeeCollector();  
}
```

ERC4626 vault implementation for LP accounting and fee distribution

Fee Vault

Vault contract: 0x4A679253410272dd5232B3F7cF5dbB88f295319

Collateral token: 0x3Aa5ebB10DC797CAC828524e59A333d0A371443c

Your collateral balance: 990

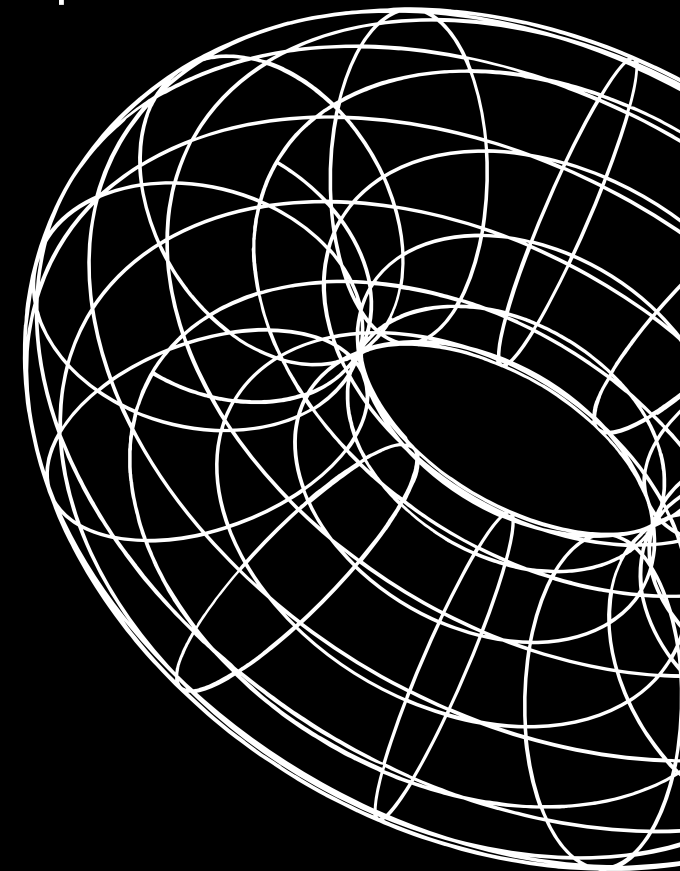
Your vault shares: 10

Total vault assets: 10

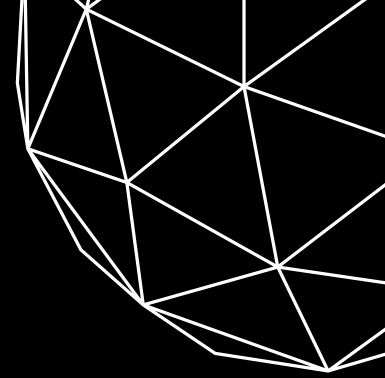
Frontend vault interaction panel with deposit and redeem functionality

```
function swap(uint256 marketId, bool buyYes, uint256 amountIn, uint256 minAmountOut)  
    external  
    virtual  
    nonReentrant  
    returns (uint256 amountOut)  
{  
    return _swap(marketId, buyYes, amountIn, minAmountOut);  
}
```

Prediction market swap and liquidity management logic



ORACLE + SUBGRAPH + FRONTEND



CHAINLINK ORACLE

- External price validation
- Staleness checks
- Oracle abstraction layer

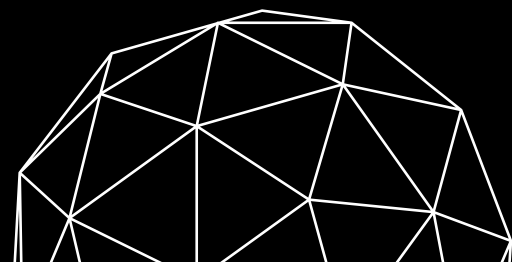
THE GRAPH

Indexed entities:

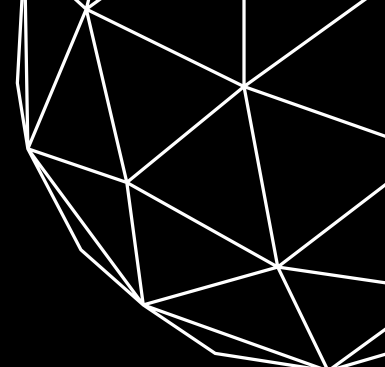
- Market
- Proposal
- Vote
- VaultFee

FRONTEND FEATURES

- MetaMask integration
- Network detection
- Error handling
- Governance UI
- Subgraph queries



ORACLE + SUBGRAPH + FRONTEND



```
if (block.timestamp - _updatedAt > stalenessThreshold) {  
    revert StalePrice(_updatedAt, stalenessThreshold, block.timestamp);  
}
```

Chainlink oracle integration with stale price validation

```
type Proposal @entity(immutable: false) {  
    id: ID!  
    proposer: Bytes!  
    description: String!  
    createdAtBlock: BigInt!  
    createdAtTimestamp: BigInt!  
}  
  
type Vote @entity(immutable: false) {  
    id: ID!  
    proposalId: BigInt!  
    voter: Bytes!  
    support: Int!  
    weight: BigInt!  
    reason: String  
    createdAtBlock: BigInt!  
    createdAtTimestamp: BigInt!  
}
```

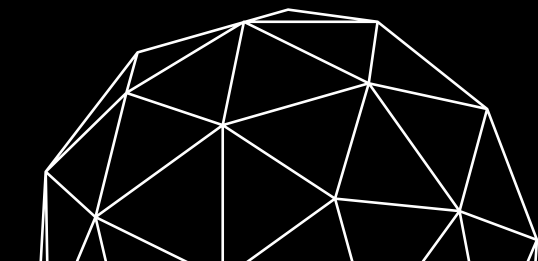
GraphQL indexed query
structure for governance and
market data

Governance Token

Connected wallet: 0xf39Fd6e51aad88F6F4ce6aB8827279cFfFb92266

Chain ID: 31337

Frontend wallet connection and network integration



SECURITY & DEVOPS

Security Engineering

- Slither static analysis
- Fuzz testing
- Invariant testing
- Reentrancy protection
- CEI pattern

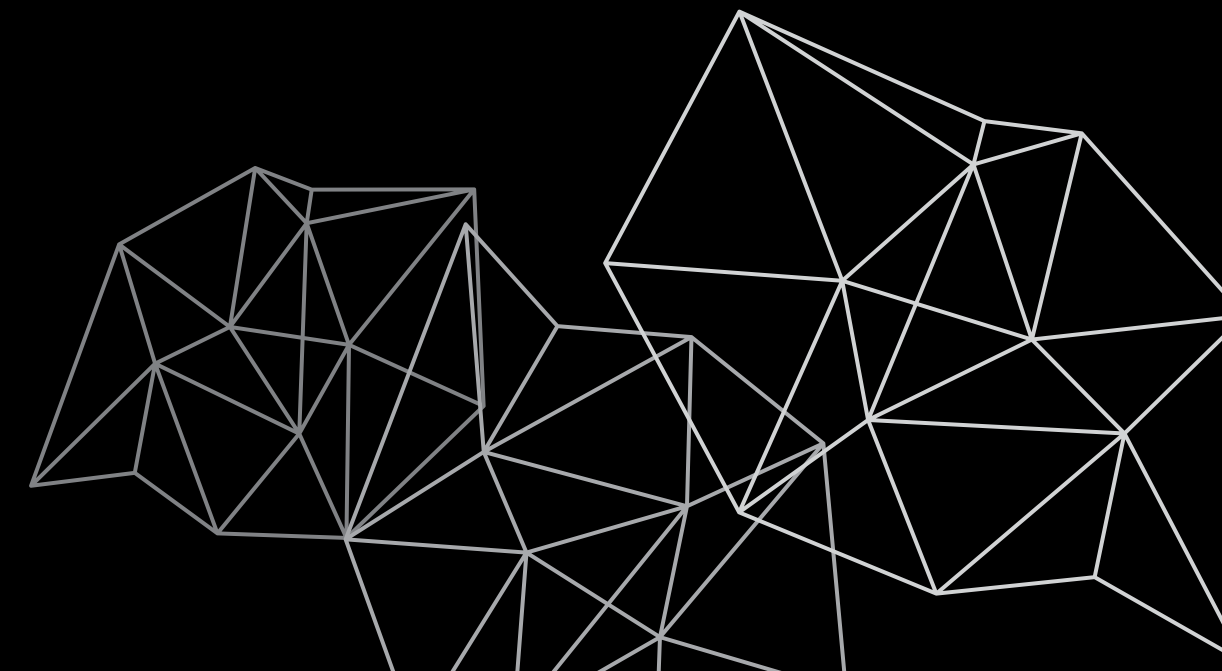
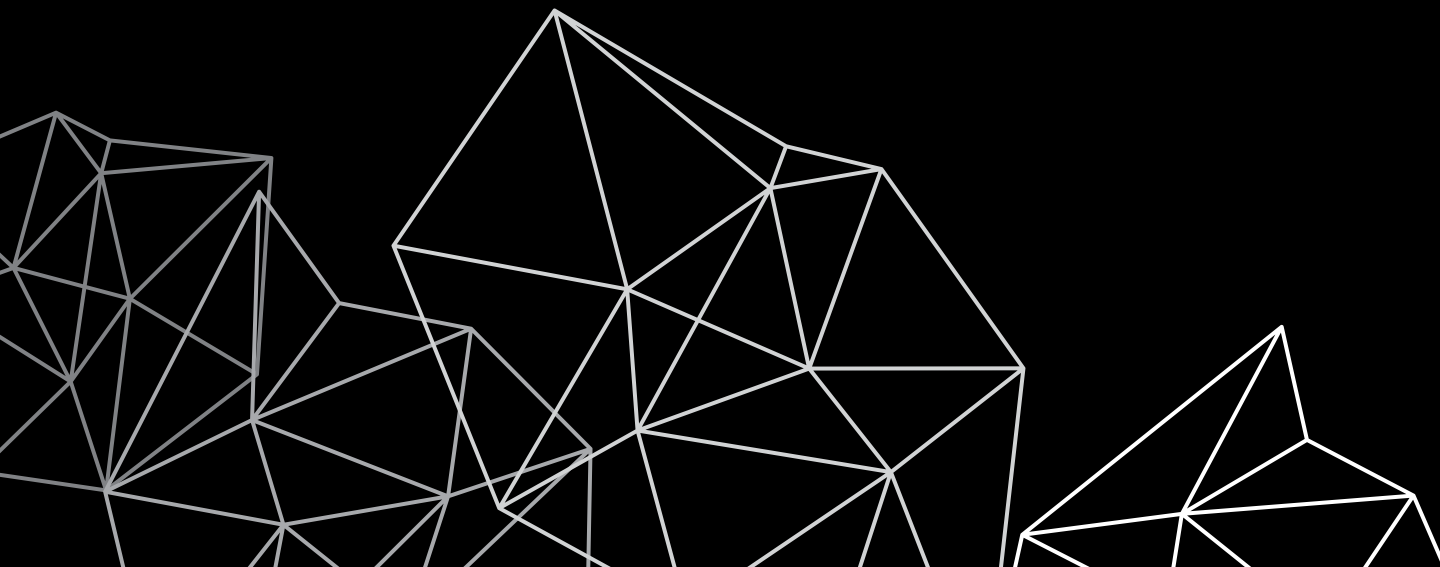
Audit Result

- 0 High findings
- 0 Medium findings

CI/CD Pipeline

GitHub Actions:

- forge build
- forge test
- forge coverage
- frontend build
- subgraph build



SECURITY & DEVOPS

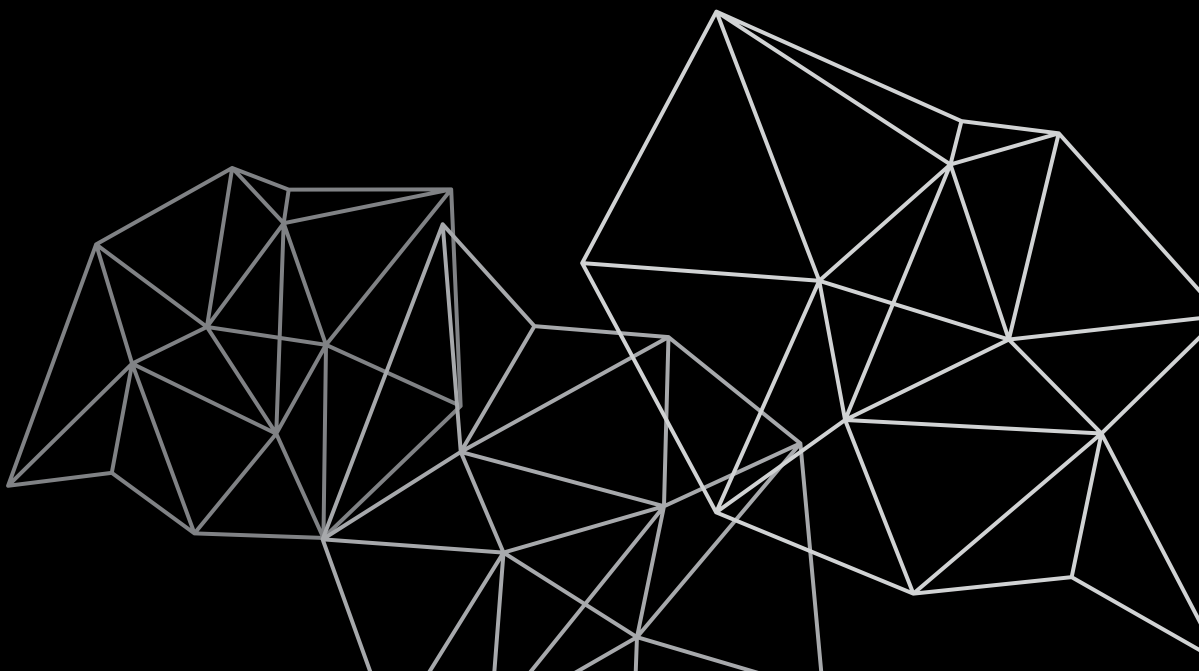
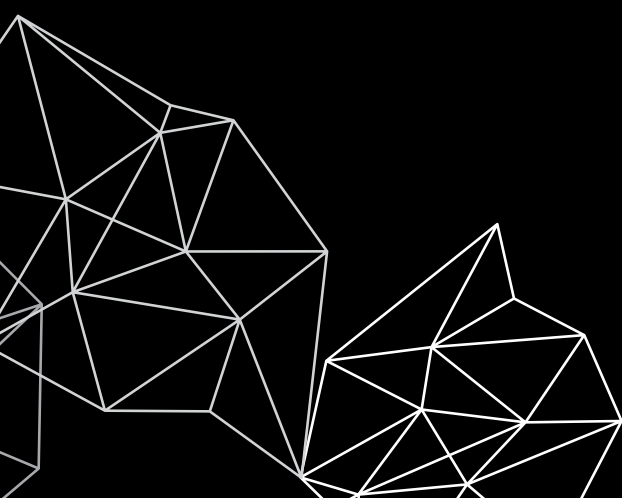
Showing runs from all workflows

8 workflow runs			Workflow ▾	Event ▾	Status ▾	Branch ▾	Actor ▾
✓	docs: add architecture, audit, deployment, frontend, gas and subgraph...	master					
CI #7: Commit f578ec8 pushed by shaidyo					May 17, 11:45 PM GMT+6		...
					7m 29s		

GitHub Actions CI/CD pipeline with successful automated checks

test
fork
ForkTests.t.sol
vulnerable
Exploits.t.sol
Mitigations.t.sol
FeeVault.t.sol
Governance.t.sol
GovernanceToken.t.sol
MarketFactory.t.sol
OracleResolver.t.sol
OutcomeShareToken.t.sol
PredictionMarketV1.t.sol
PredictionMarketV2.t.sol
YulMath.t.sol

Comprehensive smart contract testing structure



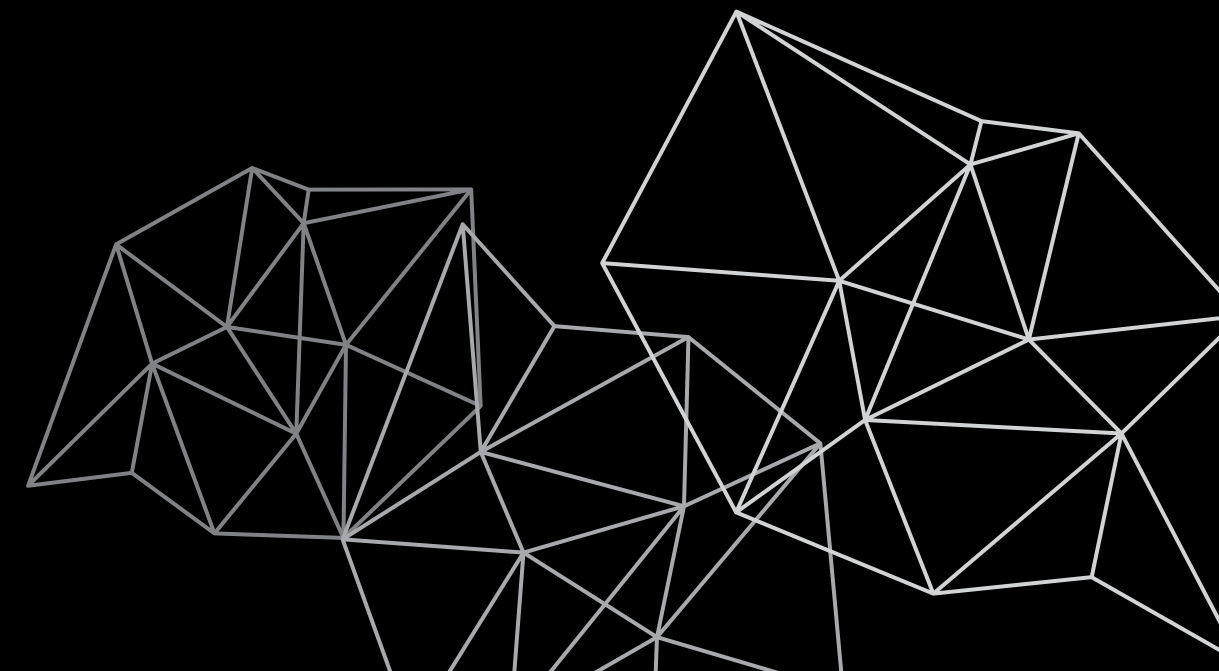
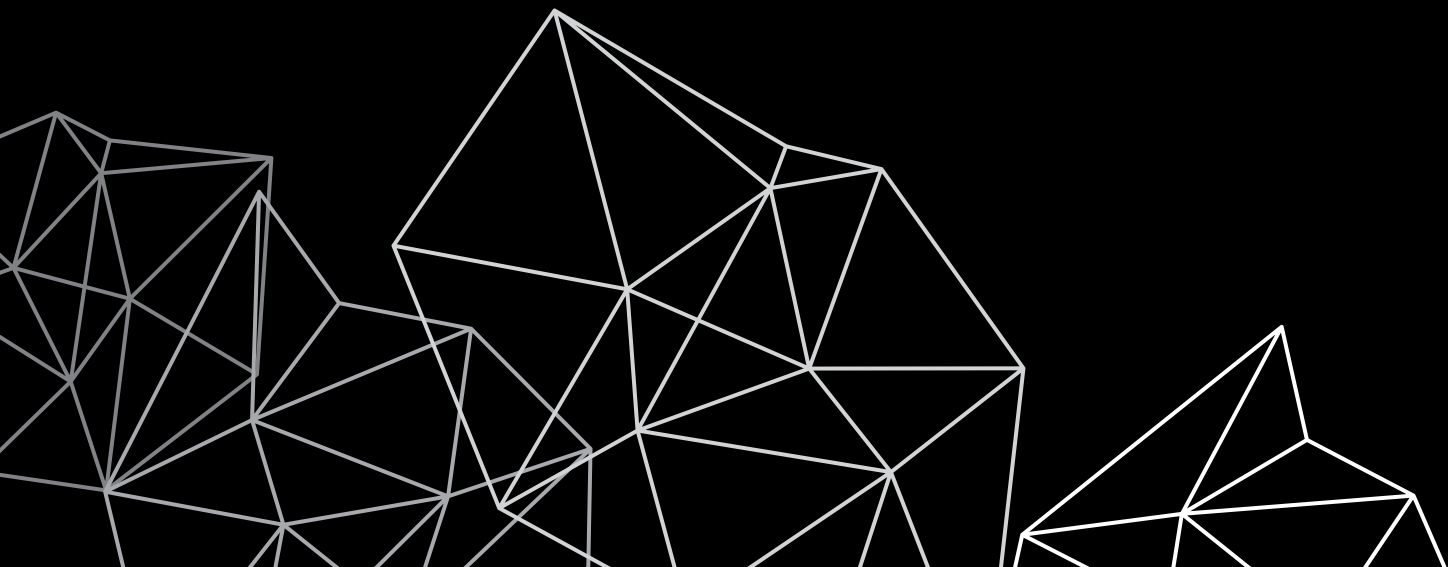
SMART CONTRACT DESIGN PATTERNS

Implemented Patterns

Why They Matter

- Factory Pattern
- UUPS Proxy Pattern
- Checks-Effects-Interactions
- Reentrancy Guard
- Access Control
- Timelock
- Oracle Adapter
- State Machine

- safer upgrades
- modular deployment
- governance security
- reentrancy protection
- deterministic architecture



SMART CONTRACT DESIGN PATTERNS

```
// — CREATE deployment —————

/// @notice Deploy a new market proxy using the EVM CREATE opcode.
///      Address is non-deterministic (depends on factory nonce).
/// @return market Address of the newly deployed proxy.
function deployWithCreate(address marketOwner) external onlyOwner returns (address market) {
    bytes memory initData = _buildInitData(marketOwner);

    // `new ERC1967Proxy(...)` compiles to a CREATE opcode
    ERC1967Proxy proxy = new ERC1967Proxy(implementation, initData);
    market = address(proxy);

    _register(market);
    emit MarketDeployedCreate(market, allMarkets.length - 1);
}

// — CREATE2 deployment —————

/// @notice Deploy a new market proxy using the EVM CREATE2 opcode.
///      Address is fully deterministic: keccak256(0xff ++ factory ++ salt ++ keccak256(initcode)).
/// @param salt      Arbitrary 32-byte value chosen by the caller.
/// @param marketOwner Owner of the deployed market.
/// @return market    Address of the newly deployed proxy.
function deployWithCreate2(bytes32 salt, address marketOwner) external onlyOwner returns (address market) {
    if (create2Market[salt] != address(0)) revert SaltAlreadyUsed(salt);

    bytes memory initData = _buildInitData(marketOwner);

    // `new ERC1967Proxy{salt: salt}{...}` compiles to a CREATE2 opcode
    ERC1967Proxy proxy = new ERC1967Proxy{salt: salt}{implementation, initData};
    market = address(proxy);

    create2Market[salt] = market;
    _register(market);
    emit MarketDeployedCreate2(market, salt, allMarkets.length - 1);
}
```

Factory pattern implementation using CREATE and CREATE2 deployment

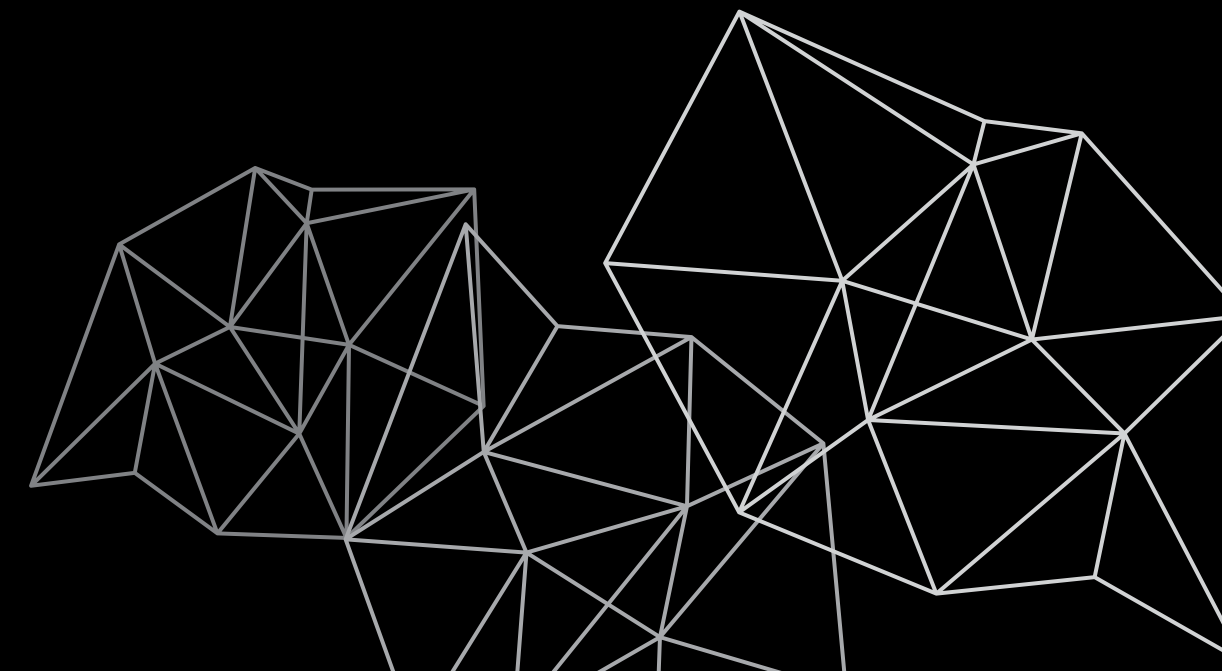
```
/// @notice Redeem winning shares for collateral at 1:1.
function redeem(uint256 marketId, uint256 shares) external nonReentrant {
    Market storage mkt = _requireMarketExists(marketId);
    if (mkt.outcome == Outcome.Unresolved) revert MarketNotResolved(marketId);
    if (mkt.outcome == Outcome.Invalid) revert InvalidOutcome();

    bool winnerIsYes = (mkt.outcome == Outcome.Yes);
    uint256 tokenId = winnerIsYes ? shareToken.yesId(marketId) : shareToken.noId(marketId);
```

Reentrancy protection using nonReentrant modifier

```
// CEI: emit before external call
emit ETHReleased(to, amount);
```

Checks-Effects-Interactions and access control implementation



TESTING STRATEGY

Implemented Testing Types

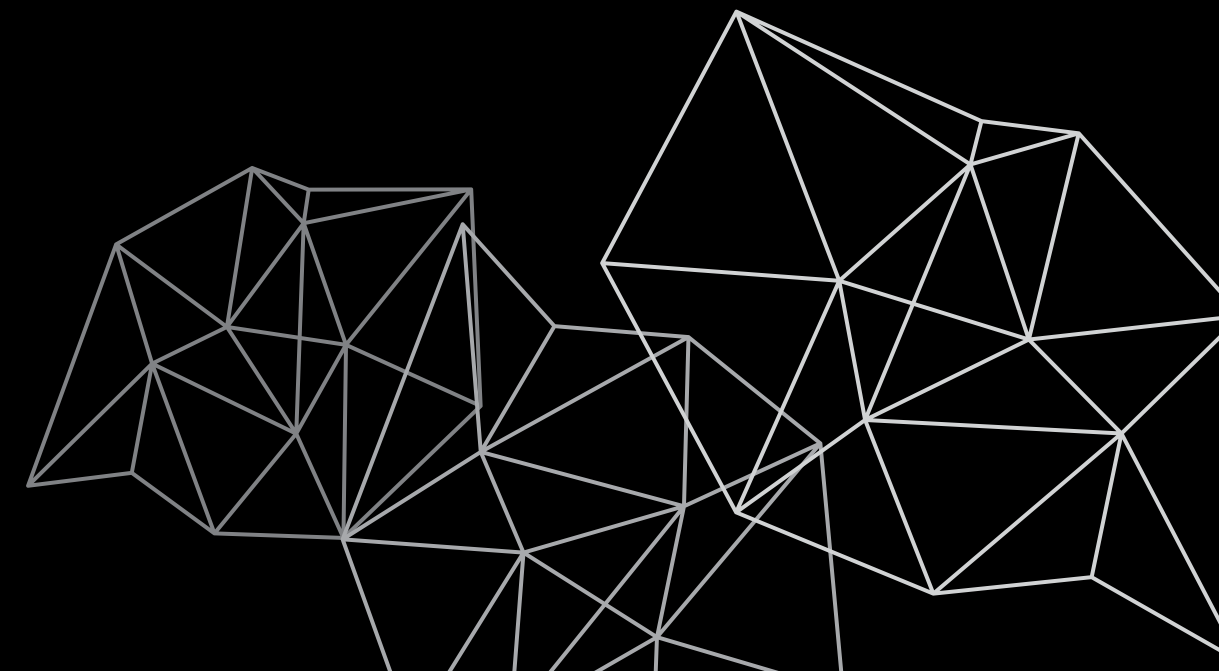
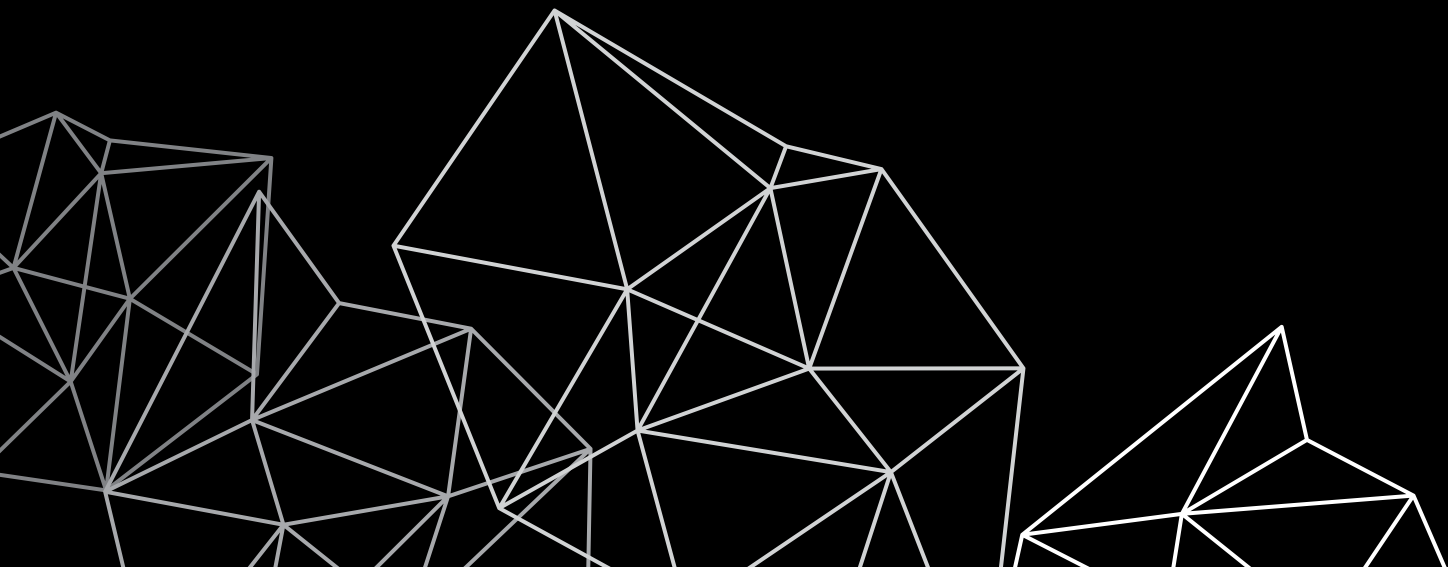
- Unit Tests
- Fuzz Tests
- Invariant Tests
- Integration Tests

Security Focus

- governance safety
- vault accounting
- swap correctness
- upgrade safety
- oracle validation

Tools Used

- Foundry
- forge test
- forge coverage
- Slither



TESTING STRATEGY

Ran 20 test suites in 49.43s (147.47s CPU time): 228 tests passed, 0 failed, 14 skipped (242 total tests)

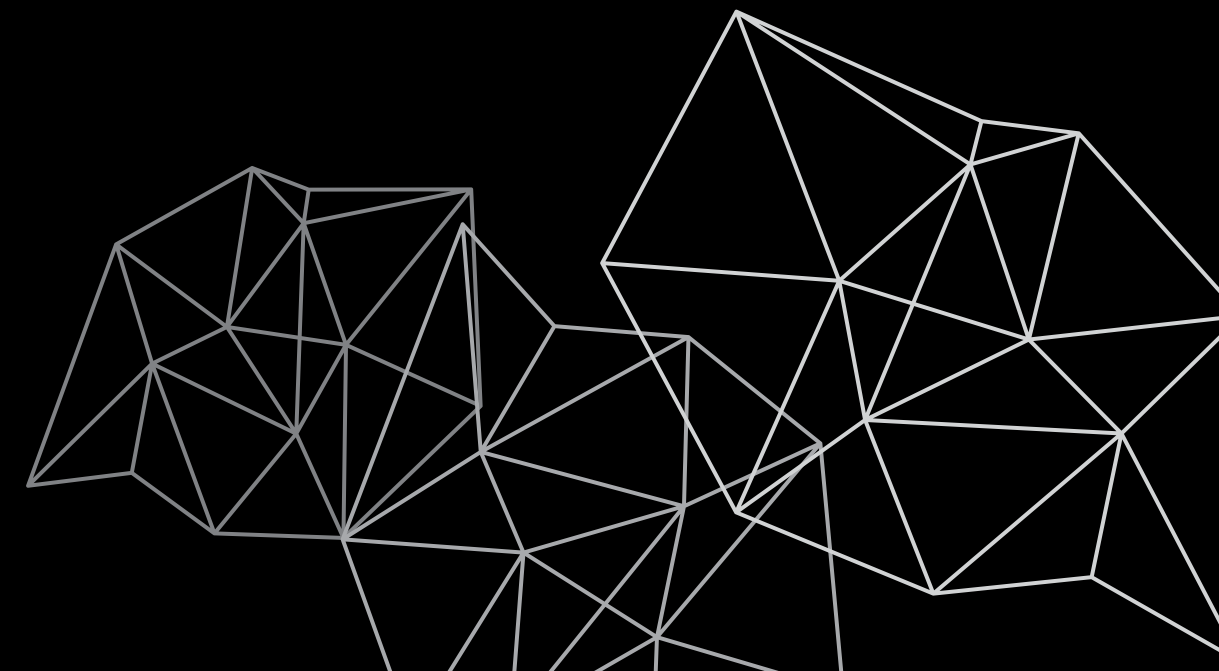
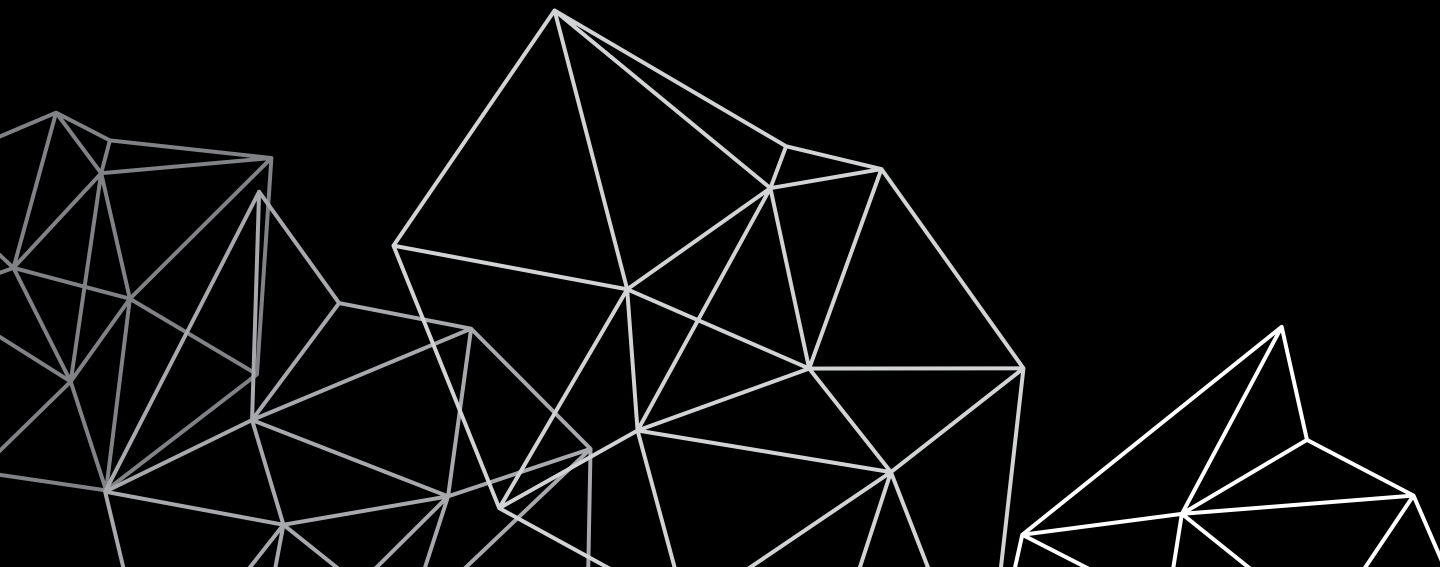
Forge unit testing execution with successful test suite results

```
/// @dev Any amount that keeps totalSupply ≤ MAX_SUPPLY must s
function testFuzz_mintWithinCap(uint256 amount) public {
    uint256 remaining = MAX_SUPPLY - token.totalSupply();
    amount = bound(amount, 0, remaining);

    vm.prank(owner);
    token.mint(alice, amount);

    assertEq(token.balanceOf(alice), amount);
    assertLe(token.totalSupply(), MAX_SUPPLY);
}
```

Fuzz and invariant testing for protocol safety verification



The background is a dark gradient with white, wavy, wireframe-like lines that create a sense of depth and movement, framing the central text.

THANK YOU